

INT234 : PREDICTIVE ANALYTICS
PROJECT REPORT
(Project Semester October 2025 -January 2026)

Credit Risk Assessment and Loan Approval Analysis

Submitted by

P.L.Geethika

Registration No: 12317641

Section: KM003

Course Code: INT 234

Under the Guidance of

Dr. Mrinalini Rana
22138

Discipline of CSE/IT

Lovely School of Computer Science

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that P.L.Geethika bearing Registration no. 12317641 has completed INT 234 project titled, “*Credit Risk Assessment and Loan Approval Analysis*” under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor

Designation of the Supervisor

School of Computer Science.

Lovely Professional University

Phagwara, Punjab.

Date: 14-12-2025

DECLARATION

I, P.L.Geethika, student of B.Tech Computer Science and Engineering under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: 14-11-2025

Signature P.L.Geethika

Registration No.12317641

1. Introduction

Loan approval is one of the most important decision-making processes in the banking and financial sector. Financial institutions must carefully evaluate customer profiles to minimize risk and avoid loan defaults. Traditionally, loan approval decisions were made manually based on predefined rules, which often resulted in delays and inconsistent decisions.

With the availability of large volumes of customer data, data science techniques can be used to automate and improve the loan approval process. Machine learning models analyze historical loan data to identify patterns that help predict whether a loan should be approved or rejected. This project aims to apply supervised learning techniques to build an efficient loan approval prediction system and compare different classifiers to determine the best-performing model.

The project involves exploratory data analysis (EDA), data preprocessing, feature scaling, and the implementation of multiple classification algorithms including K-Nearest Neighbors (KNN), Support Vector Machine (SVM), Naive Bayes, and Decision Tree. Model performance is evaluated using accuracy, confusion matrices, and classification reports, supported by advanced visualizations for better interpretability. The final outcome includes a comparative analysis of models to identify the best-performing classifier for predicting treatment improvement.

2. Source of dataset

The dataset used in this project is sourced from **Kaggle**, a well-known open platform for data science and machine learning resources. The dataset is titled “*Loan Approval Analysis*” with 50k Rows and 22 columns and is publicly available at:

 https://www.kaggle.com/datasets/parthpatel2130/realistic-loan-approval-dataset-us-and-canada?select=Loan_approval_data_2025.csv

The dataset includes attributes such as applicant income, co-applicant income, loan amount, loan term, credit history, and property details. The target variable, **Loan Status**, indicates whether the loan was approved or not. This dataset is suitable for classification problems and helps simulate real banking decision processes.. The data was downloaded in CSV format and used solely for academic and educational purposes. Prior to analysis, the dataset underwent preprocessing steps such as handling categorical variables, normalization, and feature scaling to make it suitable for machine learning model training and evaluation.

3. EDA process

Exploratory Data Analysis (EDA) is an essential step in understanding the structure, quality, and characteristics of the dataset before applying machine learning models. In this project, EDA was performed to gain insights into the data, identify patterns, and detect potential issues such as missing values and data imbalance.

3.1 Data Inspection

The dataset was initially examined by viewing the first few records using the `head()` function and reviewing the dataset structure using the `info()` function. This step helped in understanding the total number of records, feature names, data types, and the presence of both categorical and numerical variables. It also provided insights into missing values within the dataset.

3.2 Statistical Summary

Descriptive statistics were generated using the `describe()` function to analyze numerical features such as applicant income, co-applicant income, loan amount, and loan term. This analysis provided information about the mean, median, minimum, maximum, and standard deviation of each numerical attribute, helping to understand data distribution and variability.

3.3 Handling Categorical Variables

Several categorical features such as gender, marital status, education level, self-employment status, and property area were identified during the EDA process. These categorical variables were converted into numerical form using label encoding to make them compatible with machine learning algorithms. This transformation allowed the models to effectively learn patterns from non-numeric data.

3.4 Feature Scaling and Normalization

To ensure uniform contribution of all numerical features, feature scaling was applied. Standardization was used to transform numerical variables so that they have a mean of zero and a standard deviation of one. This step was particularly important for distance-based algorithms like K-Nearest Neighbors (KNN) and margin-based models such as Support Vector Machine (SVM), improving their performance and convergence.

3.5 Target Variable Analysis

The target variable in this project is **Loan Status**, which indicates whether a loan application was approved or rejected. This variable was converted into a binary format suitable for classification tasks. The distribution of loan approval and rejection cases was analyzed using visualizations to check class balance and understand the overall approval trends in the dataset.

Overall, the EDA process provided a strong foundation for feature selection, preprocessing, and model development, ensuring that the dataset was well-prepared for machine learning analysis.

4. Analysis on dataset

4.i Introduction

The objective of this analysis is to build an automated loan approval prediction system using supervised learning classification techniques. Multiple machine learning models were trained and evaluated to identify the most reliable classifier for real-world financial decision-making. The analysis focuses on model accuracy, stability, and interpretability.

4.ii General Description

The cleaned and preprocessed dataset was divided into training and testing sets to evaluate model performance on unseen data. Five different classification algorithms were implemented: Logistic Regression, K-Nearest Neighbors, Naive Bayes, Decision Tree, and Support Vector Machine (SVM). Each model was trained using the same dataset and evaluated under identical conditions to ensure a fair comparison.

4.iii Specific Requirements, Functions, and Formulas

Tools and Libraries

- Python
- Pandas and NumPy (Data manipulation)
- Matplotlib and Seaborn (Visualization)
- Scikit-learn (Model building and evaluation)

Algorithms Implemented

- Logistic Regression
- KNN
- Naive Bayes
- Decision Tree
- Support Vector Machine (SVM)

Evaluation Metrics

Accuracy

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

Precision

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

Recall

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

F1-Score

$$\text{F1-Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Confusion Matrix

A confusion matrix summarizes correct and incorrect predictions made by the model.

4.iv Analysis Results

After evaluating all classifiers, it was observed that different models showed varying performance levels. Logistic Regression and Naive Bayes performed reasonably well but were limited in handling complex data patterns. KNN and Decision Tree showed moderate performance but were sensitive to data variations.

The **Support Vector Machine (SVM)** model outperformed all other classifiers, achieving the **highest accuracy of 88.7%**. SVM effectively handled high-dimensional data and provided better generalization, making it the most suitable model for this loan approval dataset.

4.v Visualization

Visualizations were used extensively to support the analysis and enhance interpretability. The following visual tools were included:

- Distribution plots for loan approval status
- Feature-wise comparison charts
- Correlation heatmap
- Confusion matrix heatmaps for all classifiers

These visualizations helped in identifying patterns, understanding feature relationships, and evaluating model performance clearly.

5. Conclusion

This project demonstrates the practical application of data science techniques in solving a real-world financial problem. By analyzing loan applicant data and applying multiple supervised learning models, an effective loan approval prediction system was developed. Among all classifiers, **Support Vector Machine (SVM)** achieved the highest accuracy of **88.7%**, making it the most reliable model for this dataset. The results highlight the importance of data-driven decision-making in the banking sector.

6. Future scope

The current project can be further enhanced in several ways. Advanced machine learning models such as Random Forest, Gradient Boosting, and Neural Networks can be implemented to improve prediction accuracy and model robustness. Hyperparameter tuning techniques like Grid Search and Random Search can be used to optimize model performance.

The scope of this project can be extended in several ways:

- Integration of larger and real-time banking datasets
- Implementation of ensemble models for improved accuracy
- Deployment of the model as a web-based loan approval system
- Inclusion of additional financial and behavioral features
- Integration with banking decision support systems

7. References

1. Patel,P.*Loan approval*.Kaggle.
https://www.kaggle.com/datasets/parthpatel2130/realistic-loan-approval-dataset-us-and-canada?select=Loan_approval_data_2025.csv
2. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011).
Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
3. McKinney,W.(2010).
Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51–56.
4. Hunter,J.D.(2007).
Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95.
5. Waskom,M.L.(2021).
Seaborn: Statistical data visualization. Journal of Open Source Software, 6(60), 3021.
6. Géron,A.(2019).
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow (2nd ed.). O'Reilly Media.

8. Code Snippet


```
[1]: #LIBRARIES
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, precision_score, confusion_matrix, classification_report
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

```
[2]: #LOAD THE DATASET
df=pd.read_csv("C:\env\Loan_approval_data.csv")
print(df.head())
```

	customer_id	age	occupation_status	years_employed	annual_income
0	CUST100000	40	Employed	17.2	25579
1	CUST100001	33	Employed	7.3	43087
2	CUST100002	42	Student	1.1	20840
3	CUST100003	53	Student	0.5	29147
4	CUST100004	32	Employed	12.5	63657

	credit_score	credit_history_years	savings_assets	current_debt
0	692	5.3	895	10820
1	627	3.5	169	16550
2	689	8.4	17	7852
3	692	9.8	1480	11603
4	630	7.2	209	12424

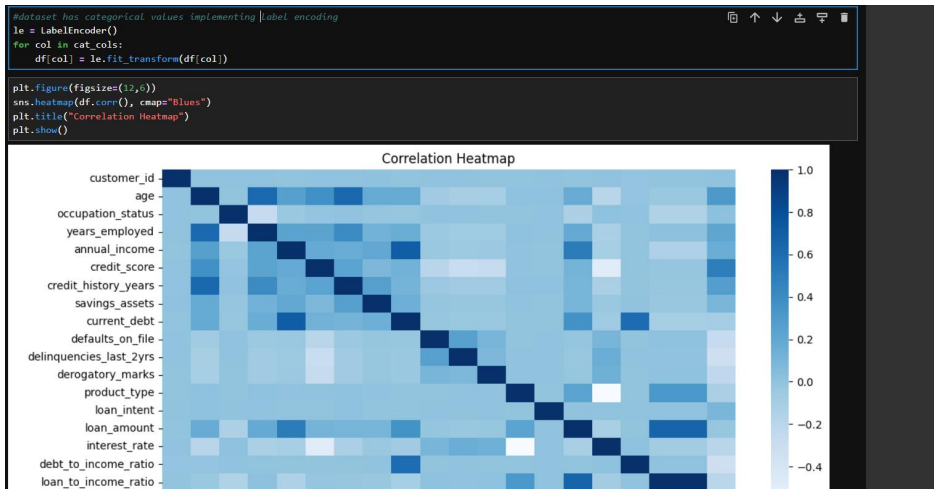
	defaults_on_file	delinquencies_last_2yrs	derogatory_marks	product_type
0	0	0	0	Credit Card
1	0	1	0	Personal Loan
2	0	0	0	Credit Card
3	0	1	0	Credit Card
4	0	0	0	Personal Loan

```
[3]: print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 20 columns):
#   Column                Non-Null Count  Dtype
---  -
0   customer_id           50000 non-null  object
1   age                   50000 non-null  int64
2   occupation_status     50000 non-null  object
3   years_employed        50000 non-null  float64
4   annual_income         50000 non-null  int64
5   credit_score          50000 non-null  int64
6   credit_history_years   50000 non-null  float64
7   savings_assets        50000 non-null  int64
8   current_debt          50000 non-null  int64
9   defaults_on_file      50000 non-null  int64
10  delinquencies_last_2yrs 50000 non-null  int64
11  derogatory_marks       50000 non-null  int64
12  product_type          50000 non-null  object
13  loan_intent            50000 non-null  object
14  loan_amount           50000 non-null  int64
15  interest_rate         50000 non-null  float64
16  debt_to_income_ratio   50000 non-null  float64
17  loan_to_income_ratio   50000 non-null  float64
18  payment_to_income_ratio 50000 non-null  float64
19  loan_status           50000 non-null  int64
dtypes: float64(6), int64(10), object(4)
memory usage: 7.6+ MB
None
```

```
[6]: print(df.nunique())
```

customer_id	50000
age	53
occupation_status	3
years_employed	395
annual_income	35770
credit_score	432
credit_history_years	301
savings_assets	10382
current_debt	25350
defaults_on_file	2
delinquencies_last_2yrs	10
derogatory_marks	5
product_type	3
loan_intent	6
loan_amount	996
interest_rate	1687
debt_to_income_ratio	796
loan_to_income_ratio	1992
payment_to_income_ratio	665
loan_status	2
dtype:	int64



```
[11]: #Train Test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

[12]: scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

[13]: from sklearn.linear_model import LogisticRegression

lr = LogisticRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
acc_lr = accuracy_score(y_test, y_pred_lr)*100

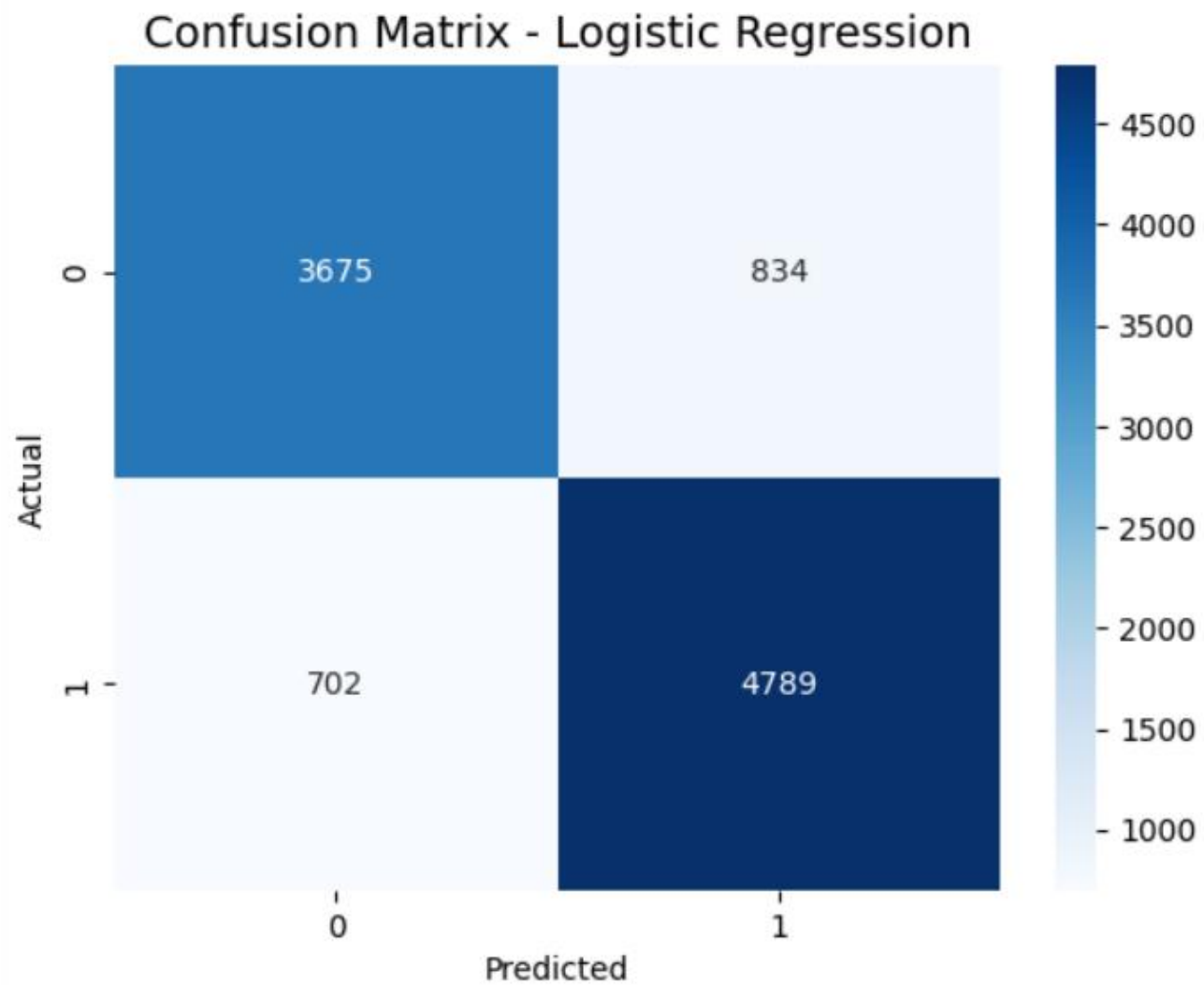
print("Logistic Regression Accuracy:", acc_lr)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_lr))
print("Classification Report:\n", classification_report(y_test, y_pred_lr))
```

```
Logistic Regression Accuracy: 84.64
Confusion Matrix:
[[3675  834]
 [ 702 4789]]
Classification Report:
              precision    recall  f1-score   support

     0       0.84       0.82       0.83       4509
     1       0.85       0.87       0.86       5491

 accuracy       0.85
 macro avg       0.85
weighted avg       0.85
```

```
#Logistic Regression
cm_lr = confusion_matrix(y_test, y_pred_lr)
sns.heatmap(cm_lr, annot=True, fmt="d", cmap="Blues")
plt.title("Confusion Matrix - Logistic Regression", fontsize=14)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```

from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
y_pred_knn = knn.predict(X_test)
acc_knn = accuracy_score(y_test, y_pred_knn)*100

print("KNN Accuracy:", acc_knn)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_knn))
print("Classification Report:\n", classification_report(y_test, y_pred_knn))

```

KNN Accuracy: 85.54

Confusion Matrix:

```
[[3606  903]
```

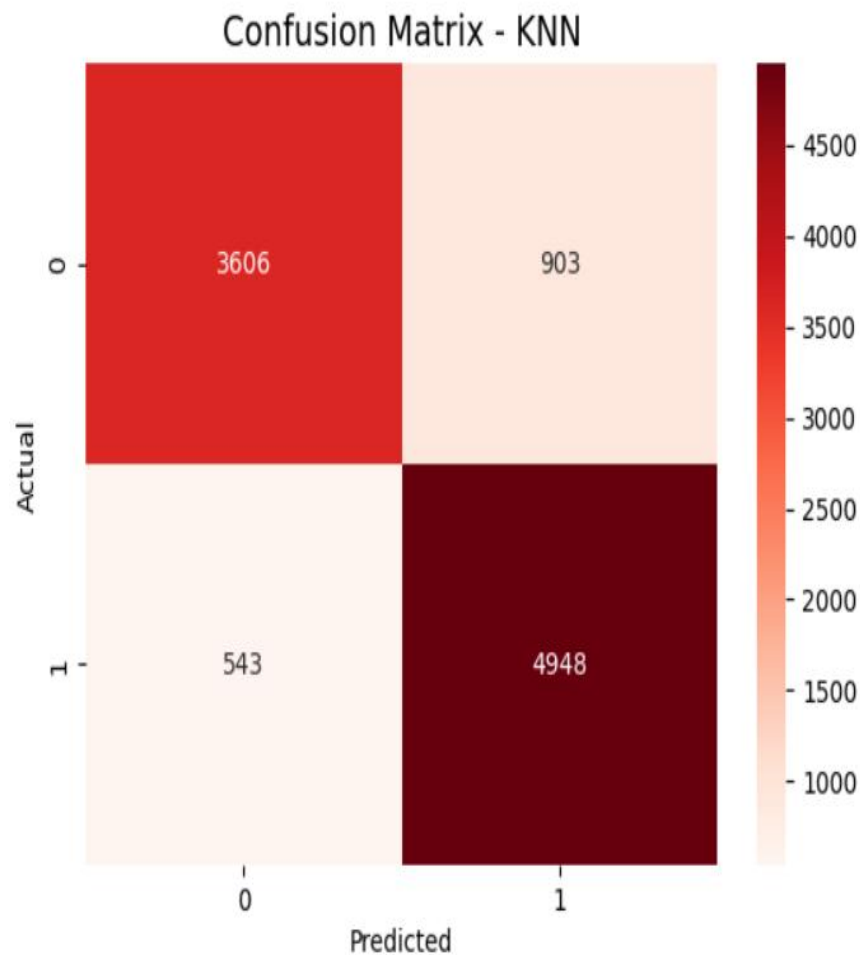
```
[ 543 4948]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.87	0.80	0.83	4509
1	0.85	0.90	0.87	5491
accuracy			0.86	10000
macro avg	0.86	0.85	0.85	10000
weighted avg	0.86	0.86	0.85	10000

```
#Confusion Matrix - KNN
```

```
cm_knn = confusion_matrix(y_test, y_pred_knn)  
sns.heatmap(cm_knn, annot=True, fmt="d", cmap="Reds")  
plt.title("Confusion Matrix - KNN", fontsize=14)  
plt.xlabel("Predicted")  
plt.ylabel("Actual")  
plt.show()
```



```
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier()
dt.fit(X_train, y_train)
y_pred_dt = dt.predict(X_test)
acc_dt = accuracy_score(y_test, y_pred_dt)*100
```

```
print("Decision Tree Accuracy:", acc_dt)
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("Classification Report:\n", classification_report(y_test, y_pred_dt))
```

Decision Tree Accuracy: 87.48

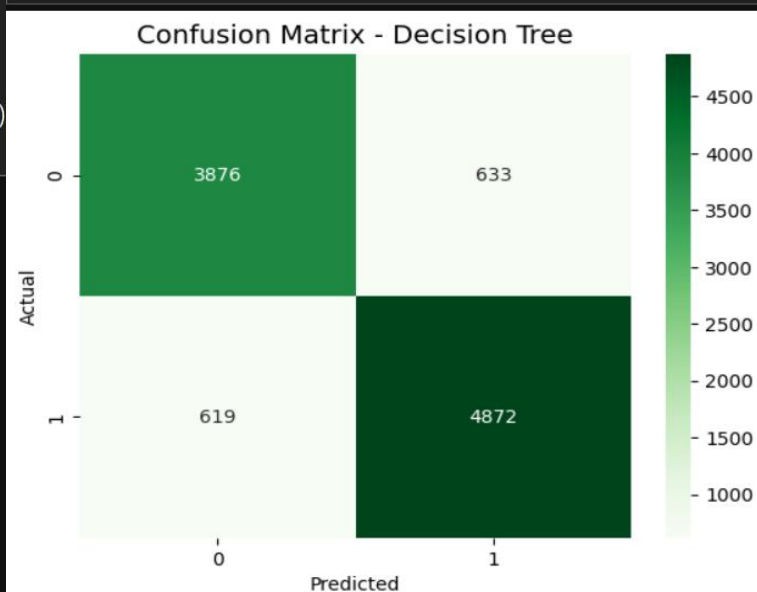
Confusion Matrix:

```
[[3876 633]
 [ 619 4872]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.86	0.86	0.86	4509
1	0.89	0.89	0.89	5491
accuracy			0.87	10000
macro avg	0.87	0.87	0.87	10000
weighted avg	0.87	0.87	0.87	10000

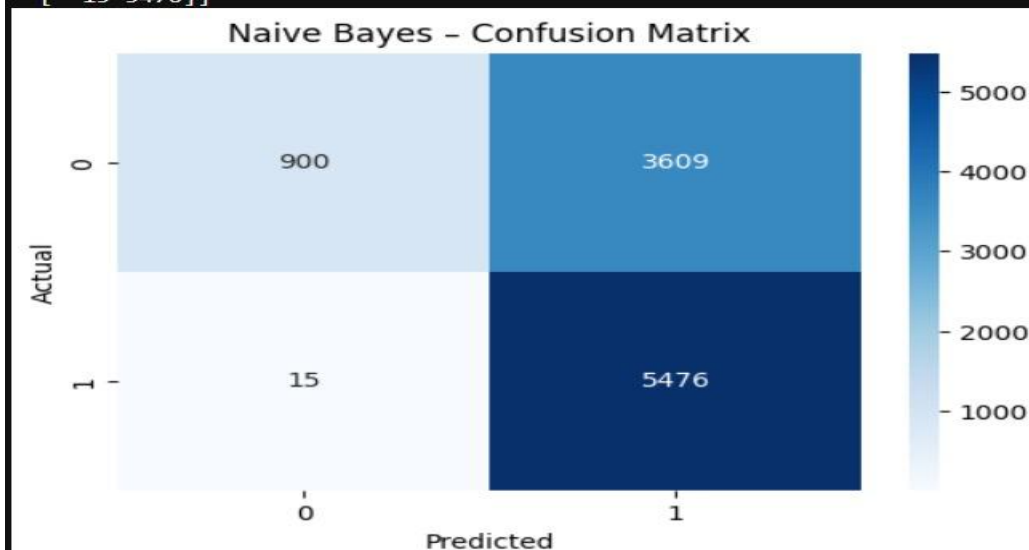
```
# Confusion Matrix - Decision Tree
cm_dt = confusion_matrix(y_test, y_pred_dt)
sns.heatmap(cm_dt, annot=True, fmt="d", cmap="Greens")
plt.title("Confusion Matrix - Decision Tree", fontsize=14)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```



```
cm = confusion_matrix(y_test, y_pred_nb)
print("Confusion Matrix:\n", cm)
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues")
plt.title("Naive Bayes - Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

Confusion Matrix:

```
[[ 900 3609]
 [  15 5476]]
```




```

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report
# Create and train the model
svm_model = SVC(kernel='rbf', probability=True, random_state=42)
svm_model.fit(X_train, y_train)
# Predictions
y_pred_svm = svm_model.predict(X_test)
# Evaluation
accuracy_svm = accuracy_score(y_test, y_pred_svm)*100
print(classification_report(y_test, y_pred_svm))
print(accuracy_svm)

```

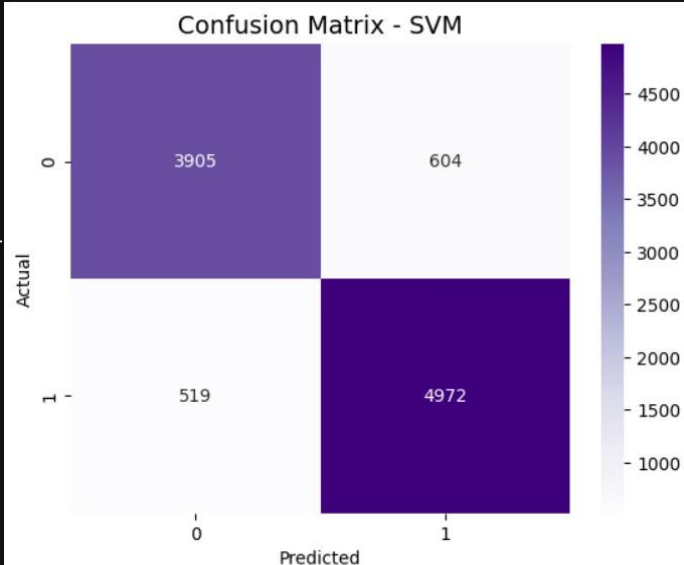
	precision	recall	f1-score	support
0	0.88	0.87	0.87	4509
1	0.89	0.91	0.90	5491
accuracy			0.89	10000
macro avg	0.89	0.89	0.89	10000
weighted avg	0.89	0.89	0.89	10000

88.77000000000001

```

# svm visualization
cm_svm = confusion_matrix(y_test, y_pred_svm)
sns.heatmap(cm_svm, annot=True, fmt="d", cmap="Purples")
plt.title("Confusion Matrix - SVM", fontsize=14)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

```



LinkedIn:-www.linkedin.com/in/pamidi-geethika34

GitHub:-<https://github.com/Geethika-pamidi/Loan-Approval-Prediction-System>